# Message-Oriented Middleware: MQTT and AMQP

Communication Technologies for the Internet of Things, Lecture 12

Catarina Silva

ESTGA, Universidade de Aveiro | 2026

# Agenda (1/2)

- The message-oriented paradigm and the motivation for decoupling
- Distribution models: queues and publish-subscribe
- AMQP: model, characteristics and the benefits of being an open standard
- MQTT: model, quality-of-service levels and features
- Comparing AMQP and MQTT

# Agenda (2/2)

- Implementations and adoption
- Selection criteria

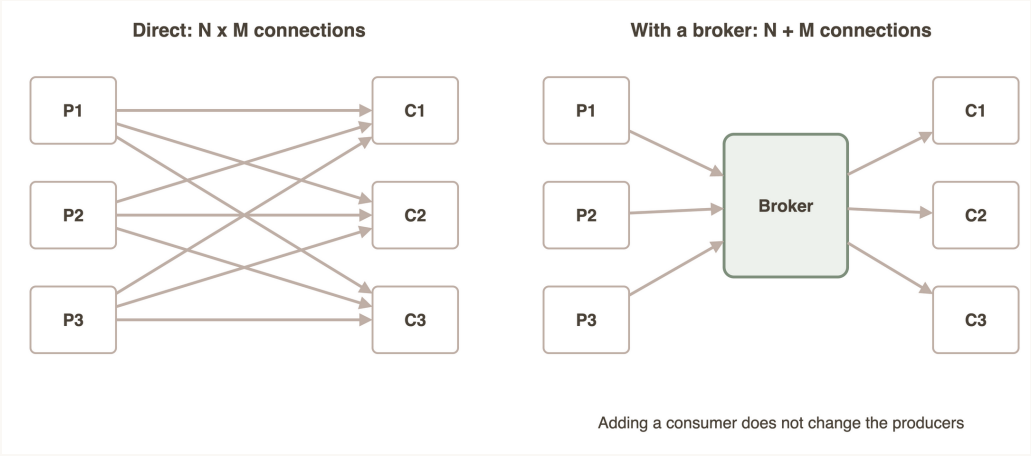
# Motivation: Scaling Requires Flexibility (1/2)

- The scale of a solution imposes flexibility in its architecture: what works with ten sensors rarely works with ten thousand.
- Conventional programming models, in which components call each other directly, scale very poorly.
- Cloud computing offers remarkable dynamic scaling, but applications have to be built so as to take advantage of that model.
- The messaging model decouples components, and it is that decoupling that makes it possible to scale up.

# Motivation: Scaling Requires Flexibility (2/2)

- Most cloud infrastructures already integrate message-oriented middleware, such as RabbitMQ.
- Concretely: whoever produces the data no longer needs to know who consumes it, how many consumers there are, or whether they are even available at the moment of publication.

# Direct or Through a Broker



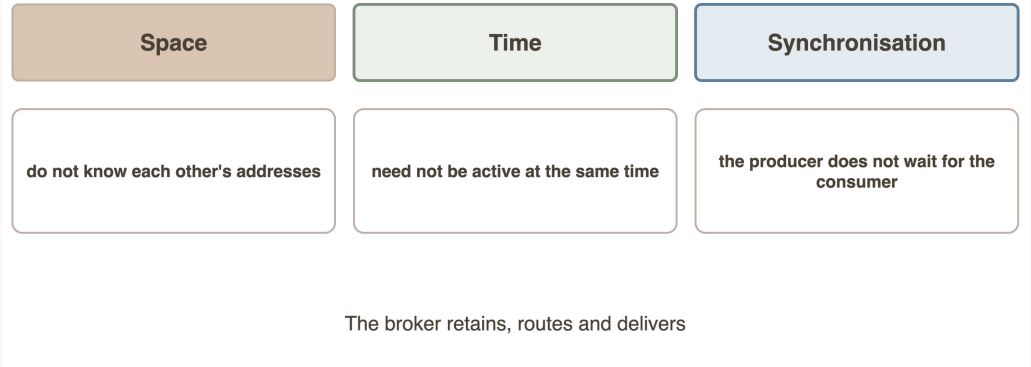
# What Message-Oriented Middleware Is (1/2)

- It is a layer of software placed between data producers and consumers, taking responsibility for routing messages.
- The central element is the *broker*, a server that receives messages from producers and delivers them to the appropriate consumers.
- **Space decoupling:** producer and consumer do not need to know each other's addresses.
- **Time decoupling:** they do not need to be active at the same time, since the broker retains the messages.

# What Message-Oriented Middleware Is (2/2)

- **Synchronisation decoupling:** the producer is not blocked waiting for the consumer to process the message.
- In an IoT solution, this is the point where the data collected by a gateway stops being confined to the process that collected it.

# The Three Decouplings



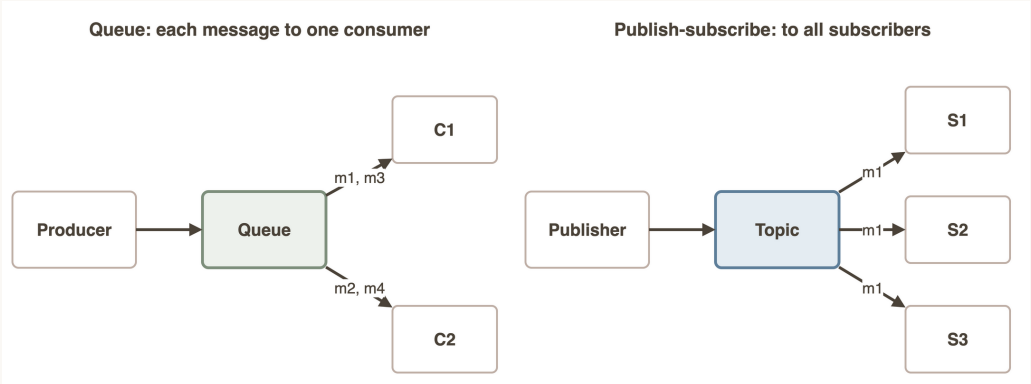
# Distribution Models (1/2)

- **Queuing model (*message queuing*):** each message is placed on a queue and delivered to a single consumer. Several consumers can take from the same queue, sharing the workload between them.
- This model suits task distribution, in which each unit of work must be processed exactly once.
- **Publish-subscribe model:** each message published on a topic is delivered to all consumers subscribed to that topic.

## Distribution Models (2/2)

- This model suits the dissemination of sensing data, where the same reading may interest an alarm mechanism, an analytics module and a storage system at once.
- AMQP supports both models; MQTT relies exclusively on publish-subscribe.
- In IoT solutions, the second model is the most frequent, precisely because it allows consumers to be added without changing producers.

# Queues and Publish-Subscribe



# AMQP: Advanced Message Queuing Protocol (1/2)

- All resources are created and destroyed dynamically by clients, as needed: there is no static pre-configuration of the broker.
- The conceptual model is simple and clean, based on just three fundamental concepts.
- It is an open standard, developed by the AMQP working group.
- Client libraries exist for many programming languages, freely available.

# AMQP: Advanced Message Queuing Protocol (2/2)

- RabbitMQ is an open-source, free and widely used broker, which will be used in this lecture's lab.
- This combination (open standard, free implementation and libraries in several languages) explains much of the protocol's adoption.

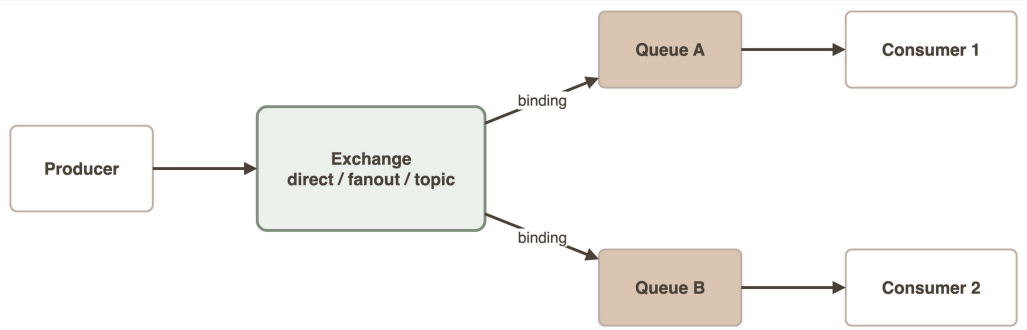
# AMQP: How It Works (1/2)

- The three central concepts are the message, the exchange type and the queue.
- The producer publishes a message, which is delivered to an exchange on the broker; it is the exchange that decides, based on its type and the configured binding rules, which queues the message should go to.
- The message is then placed on the corresponding queue or queues, where it waits until consumed.
- The protocol guarantees ordered consumption of messages within each queue.

# AMQP: How It Works (2/2)

- It allows multiple consumers to receive messages from the same topic, ensuring different consumption rates among them.
- Messages are routed between different consumers, and topics with no subscribers are discarded.
- Topics allow wildcards, which increases addressing flexibility and lets whole families of topics be subscribed to.

# From Producer to Consumer in AMQP




The exchange decides, by type and bindings, which queues the message goes to

# AMQP: Advanced Features (1/2)

- Errors can be raised when messages cannot be delivered to any subscriber, which avoids silent loss.
- Messages can be consumed in such a way that the broker retains them until the subscriber confirms receipt.
- Messages can be published indicating that they should be persisted to disk, which guarantees their survival across a broker restart.
- The protocol supports transactions in publishing and acknowledgement, in the form of atomic operations.

# AMQP: Advanced Features (2/2)

- It has flow control, used, among other purposes, to prevent a fast producer from overwhelming a slow consumer.
- This set of guarantees explains why AMQP is the usual choice in enterprise contexts, where the loss of a message may have significant consequences.

# AMQP: Benefits of Being an Open Standard (1/2)

- It facilitates interoperability between multiple implementations of the protocol.
- It avoids dependence on a single vendor: it becomes simpler to replace individual components with alternative implementations.
- It allows third parties to write client libraries for other languages, widening the reach of the protocol.

# AMQP: Benefits of Being an Open Standard (2/2)

- It allows independent traffic analysis tools to inspect and decode the interactions between clients and brokers, which is valuable for diagnosis and security audits.
- It promotes similarities between the programming interfaces of different implementations, reducing migration effort.

# AMQP: Implementations (1/2)

- **Apache Qpid:** open-source project of the Apache Foundation, which maintains support for several AMQP versions.
- **Apache ActiveMQ:** another open-source project of the Apache Foundation.
- **RabbitMQ:** open-source broker, the most widely used, and the one adopted in this course's lab.
- **Azure Event Hubs and Azure Service Bus:** managed services on Microsoft's cloud platform.
- **Solace PubSub:** multiprotocol broker, available in hardware, software and cloud.

# AMQP: Implementations (2/2)

- **JORAM:** open-source Java implementation, from the OW2 consortium.
- **StormMQ:** commercial service based on AMQP.

# MQTT: Message Queue Telemetry Transport (1/2)

- Described as a lightweight, publish-subscribe messaging protocol, for use over the TCP/IP stack.
- It follows the publish-subscribe model and requires a message broker.
- Standardised as ISO/IEC 20922.
- It is characterised by a small code footprint, which makes it implementable on devices with little memory.

# MQTT: Message Queue Telemetry Transport (2/2)

- It was designed for environments with limited bandwidth, precisely the constrained environments typical of the Internet of Things.
- It was developed by IBM in 1999 and made available with no licensing costs in 2010.
- It is data-agnostic: it imposes no format on the message payload, leaving that decision to the application.

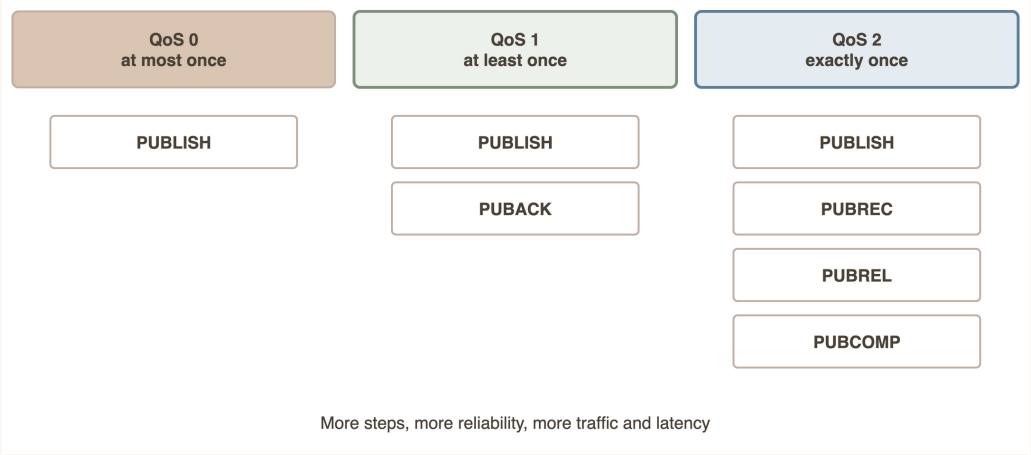
# MQTT: Quality-of-Service Levels (1/2)

- **QoS 0, at most once:** the broker or the client delivers the message a single time, with no acknowledgement. It is the lowest-cost level, suited to periodic readings where occasional loss is irrelevant.
- **QoS 1, at least once:** the message is delivered with acknowledgement required. It guarantees that the message arrives, but allows duplicate deliveries if the acknowledgement is lost.
- **QoS 2, exactly once:** the broker or the client guarantees single delivery, through a four-step protocol. It is the most reliable level and the most expensive in traffic and latency.

# MQTT: Quality-of-Service Levels (2/2)

- The level is chosen message by message, which lets the highest cost be reserved for the data that actually justifies it.
- A command that acts on equipment justifies QoS 2; a temperature reading sent every minute rarely justifies more than QoS 0.

# The Exchanges of Each QoS Level



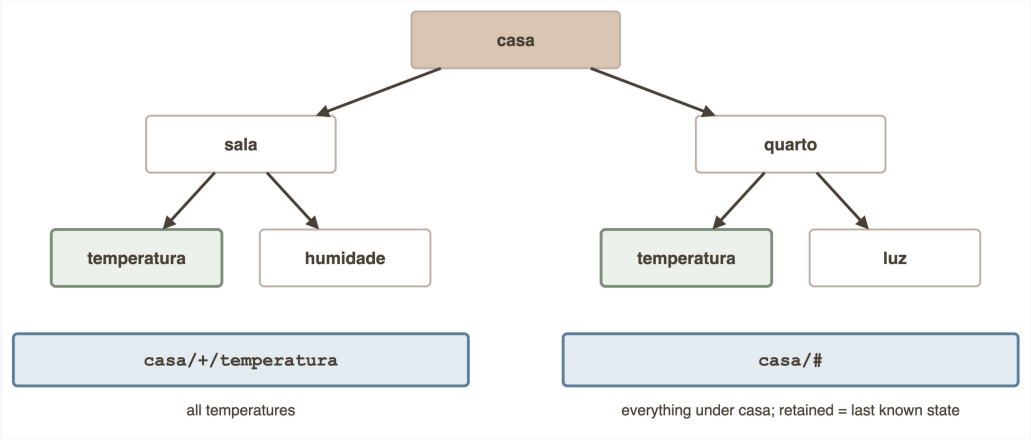
# MQTT: Retained Messages and Topics (1/2)

- The broker may retain the last message published on each topic, called a retained message.
- A client that subscribes to the topic later immediately receives that message, without waiting for the next publication.
- This feature is particularly useful when the interval between publications is long: a new consumer immediately knows the sensor's last known state.
- Topics are organised hierarchically, with levels separated by slashes, and allow wildcards in subscriptions.

# MQTT: Retained Messages and Topics (2/2)

- This organisation allows the namespace to be structured meaningfully, for example by installation, by room and by type of quantity measured.
- A consumer can thus subscribe to all the temperature sensors of a building with a single subscription.

# Topics and Wildcards



# MQTT: Adoption (1/2)

- Facebook Messenger used MQTT to carry messages in its mobile app.
- Amazon Web Services announced in 2015 the Amazon IoT service, based on MQTT.
- The SensorThings API specification, from the Open Geospatial Consortium, includes an MQTT extension, demonstrated in an IoT pilot project of the United States Department of Homeland Security.
- OpenStack's infrastructure services are interconnected by a unified MQTT message bus, with Mosquitto as the broker.

# MQTT: Adoption (2/2)

- Adafruit provides a public MQTT-based service for IoT projects.
- IECC Scalable is another example of use in control systems.
- This breadth of adoption, from mobile apps to industrial platforms, is explained by the simplicity of the protocol and its low implementation cost.

# Comparing AMQP and MQTT (1/2)

- **Complexity:** AMQP offers a richer model, with exchanges, queues and transactions; MQTT is limited to publish-subscribe over topics.
- **Overhead:** MQTT was designed to minimise protocol traffic, being better suited to low-bandwidth links.
- **Guarantees:** AMQP offers atomic transactions, flow control and persistence to disk; MQTT offers three quality-of-service levels and retained messages.
- **Supported models:** AMQP supports queues and publish-subscribe; MQTT only publish-subscribe.

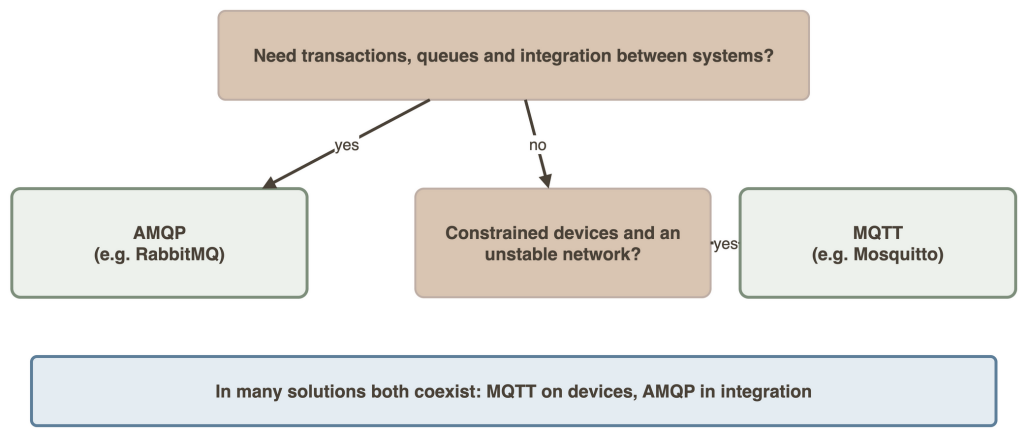
# Comparing AMQP and MQTT (2/2)

- **Focus:** AMQP is aimed at enterprise interoperability and integration between systems; MQTT is aimed at constrained devices and unstable networks.
- Published comparative studies evaluate both on unstable and mobile networks, considering message size against protocol overhead, energy consumption against resource requirements, and bandwidth against latency.

# AMQP and MQTT in a Table

| | AMQP | MQTT |
|---|---|---|
| Models | queues and pub-sub | pub-sub |
| Guarantees | transactions, disk, flow control | 3 QoS levels, retained |
| Overhead | higher | minimal |
| Focus | enterprise integration | constrained devices |

# Which to Choose



## Lecture Summary (1/2)

- Message-oriented middleware solves an architecture problem, not a transport problem: it decouples whoever produces data from whoever consumes it, in space, in time and in synchronisation.
- It is this decoupling that allows analytics, alarm or storage mechanisms to be added without changing the devices that collect the data.
- AMQP is an open standard with a rich model (exchanges, queues, transactions, flow control) aimed at enterprise integration and interoperability between implementations.

## Lecture Summary (2/2)

- MQTT is a deliberately minimalist protocol, with three quality-of-service levels and retained messages, aimed at constrained devices and low-bandwidth networks.
- Both run over TCP and require a broker; the choice depends on the relative weight given to transactional guarantees and to resource economy.
- The next lecture applies these concepts, connecting a Zigbee network to an MQTT broker and thus closing the path from sensor to middleware.

# Thank you.

*Questions and comments*